TP2

Projeto de Bloco: Ciência da Computação

Professor: Leonardo Silva da Gloria
Aluno: Bruno Müller
Turma: EDS
Atividade: TP2
Data: 06/10/2026
Repositório: https://github.com/BrunoMullerInfnet/tp2-leo.git

Ambiente

Os programas estão nas pastas Exercicio1 até Exercicio5 do projeto. Para executar usei o container da disciplina, montando a pasta do trabalho em /app:

docker run -it -u root -v "C:\Users\bruno\tp2-leo\TP2_PB_Sistemas_Operacionais-main:/app" leogloriainfnet/tp-ciencia-computacao bash

Dentro do container o código fica em /app e os arquivos de entrada ficam em /dados. Os tempos que o programa imprime mudam de uma execução para outra, então no relatório eu explico o que cada programa mede e como eu comparo, em vez de cravar um número de relógio.

Exercício 1

Parte A. Arquivo: Exercicio1/ParteA_Exer1.py

O objetivo é contar linhas e palavras de cada arquivo em /dados/arquivos sem travar o programa numa leitura só. A pasta está na constante PASTA. No main eu uso os.listdir, monto o caminho com os.path.join e descarto o que não for arquivo. Depois ordeno a lista para a saída sair sempre na mesma ordem.

Para cada caminho eu crio uma tarefa com asyncio.create_task chamando ler_um. Essa função não lê o arquivo direto no event loop. Ela manda contar para outra thread com asyncio.to_thread, porque open é bloqueante. contar abre o arquivo em utf-8 (errors="replace" para não quebrar em caractere estranho), soma 1 a cada linha e conta as palavras com linha.split(). No fim devolve o nome do arquivo, as linhas e as palavras.

asyncio.gather espera todas as tarefas. Em seguida eu imprimo, para cada arquivo, o nome, "Linhas" e "Palavras", e no final "Tempo total" com duas casas. O cronômetro é time.perf_counter e começa depois de a lista de arquivos já estar pronta, então a montagem da lista não entra na conta.

Como cada linha é lida uma vez, o trabalho cresce com o tamanho total do texto, O(N). A leitura linha a linha também evita carregar o arquivo inteiro de uma vez.

Para rodar, dentro do container:

cd /app/Exercicio1
python3 ParteA_Exer1.py

Parte B. Arquivo: Exercicio1/ParteB_Exer1.c

A parte B faz a mesma contagem, agora em C, e reparte os arquivos entre threads com OpenMP. Cada arquivo vira um item da struct Contagem, com nome, linhas e palavras. O opendir abre /dados/arquivos. No readdir eu pulo nome que começa com ponto, para não entrar "." e "..". O limite do vetor é 1000 arquivos.

ler_arquivo monta o caminho, abre com fopen e lê com getline. conta_palavras anda caractere por caractere: espaço tira a flag de "dentro da palavra", e o primeiro caractere que não é espaço soma uma palavra. É o mesmo critério de separar por espaço, só que escrito na mão.

O paralelismo está neste trecho:

#pragma omp parallel for
for (i = 0; i < qtd; i++) ler_arquivo(&lista[i]);

Cada iteração trata um arquivo. O tempo eu meço com omp_get_wtime, do fim da listagem até o fim do for. A saída repete nome, linhas e palavras, e fecha com "Tempo total" em 6 casas.

Compilação e testes pedidos, com 1, 2, 4 e 8 threads:

```
cd /app/Exercicio1
gcc -fopenmp ParteB_Exer1.c -o programa
OMP_NUM_THREADS=1 ./programa
OMP_NUM_THREADS=2 ./programa
OMP_NUM_THREADS=4 ./programa
OMP_NUM_THREADS=8 ./programa
```

O -fopenmp liga o OpenMP. OMP_NUM_THREADS diz quantas threads o for paralelo pode usar. O speedup de uma rodada é o tempo com 1 thread dividido pelo tempo daquela rodada. Eu fico com a configuração que tiver o menor tempo. Mais thread não garante ganho: criar e dividir o trabalho custa, e a leitura pode esbarrar no disco. Se o speedup passar do número de threads, o mais provável é cache, porque uma execução seguinte encontra parte dos arquivos já na memória. O trabalho lógico continua O(N). Com T threads o tempo pode se aproximar de O(N/T), limitado pelo disco, pelos núcleos e pela coordenação das threads.

Perguntas

1. Qual é a diferença entre assincronismo e paralelismo?
Assincronismo é seguir com outra tarefa enquanto uma está esperando, por exemplo uma leitura. Paralelismo é executar partes ao mesmo tempo, em threads ou núcleos diferentes. Na parte A a espera fica no asyncio. Na parte B as threads do OpenMP realmente dividem o for.

2. O uso de asyncio significa necessariamente que duas instruções estão sendo executadas simultaneamente?
Não. O asyncio alterna tarefas num event loop. Uma tarefa anda quando a outra espera. Isso é concorrência. Na minha parte A a leitura sai do loop porque eu uso to_thread, então a parte bloqueante vai para thread, mas o gather em si não quer dizer que o processador está executando duas linhas do Python ao mesmo tempo.

3. O uso de mais threads sempre torna o programa mais rápido?
Não. Thread extra também gasta tempo para ser criada e para repartir o serviço. Se o limite for o disco, ou se já não houver núcleo livre, o tempo para de cair e pode até piorar.

4. Em qual configuração você encontrou o melhor desempenho?
A melhor é a que tiver o menor "Tempo total" entre 1, 2, 4 e 8 threads. Eu calculo o speedup dividindo o tempo de 1 thread pelo tempo de cada uma. O número muda se eu rodar de novo, principalmente por causa de cache, então eu olho a saída daquela execução e não um valor fixo.

Exercício 2

Os dois programas leem inteiros de /dados/numeros.txt. Eu não usei sort nem sorted. O 1000º menor, com índice começando em 0, é a posição 999.

Parte A. Arquivo: Exercicio2/ParteA_Exer2.py

quicksort ordena o vetor no lugar. particionar escolhe o pivô no meio do pedaço, a função trocar joga esse elemento para o fim, e o for separa: quem é menor ou igual ao pivô vai para a esquerda. No fim o pivô volta para o meio da partição, na posição i + 1.

quicksort só continua se esq < dir. A chamada que o main faz não entra no contador. Cada particionamento soma 2 em "chamadas", uma para cada lado. Cada elemento olhado no for soma 1 em "comparacoes". O perf_counter cobre só o quicksort e o acesso a numeros[999], a leitura do arquivo fica de fora.

A saída é o 1000º menor, a quantidade de comparações, a quantidade de chamadas recursivas e o tempo. Em média o quicksort fica em O(n log n), porque o pedaço vai sendo dividido. Se o pivô sempre deixar um lado quase vazio, cai para $O(n^2)$.

```
cd /app/Exercicio2
python3 ParteA_Exer2.py
```

Parte B. Arquivo: Exercicio2/ParteB_Exer2.py

O quickselect usa o mesmo particionar (pivô no meio, trocar, menores à esquerda), mas é um while, não uma recursão dos dois lados. Se a posição do pivô for 999, aquele valor já é a resposta. Se o alvo estiver à esquerda, eu corto a direita. Se estiver à direita, eu corto a esquerda. O lado que não tem a posição some.

Por isso os dois programas têm que achar o mesmo número: os dois organizam o vetor com a mesma regra de pivô, e o quicksort deixa o 1000º menor exatamente na posição 999. A diferença é o trabalho. O quicksort arruma o vetor inteiro. O quickselect para quando acha a posição. A média do quickselect é O(n). O pior caso, pivô sempre ruim, também é $O(n^2)$.

python3 ParteB_Exer2.py

Na comparação eu olho os dois "Tempo de execucao". O quickselect tende a ser bem mais curto nesse problema, porque descarta um pedaço a cada volta. A escolha do pivô manda nos dois: pivô perto do centro encurta o caminho, pivô na ponta estica.

Perguntas

1. Por que ordenar todo o vetor pode representar trabalho desnecessário?
Porque a pergunta é só o valor da posição 999. O resto da ordem não precisa ficar pronto. O quickselect segue só no lado onde essa posição pode estar.

2. Qual é a complexidade média do QuickSort?
O(n log n).

3. Qual é a complexidade média do QuickSelect?
O(n).

4. Qual é o pior caso dos dois algoritmos?
$O(n^2)$, quando o pivô deixa as partições muito tortas e o algoritmo quase não reduz o pedaço.

5. Como a escolha do pivô pode influenciar o desempenho?
Pivô próximo do meio dos valores divide melhor e reduz etapas. Pivô muito pequeno ou muito grande deixa um lado enorme e empurra o algoritmo para o pior caso. Nos dois arquivos o pivô é o elemento do meio do intervalo, justamente para não pegar sempre a ponta do pedaço.

Exercício 3

Arquivo: Exercicio3/ListaDuplaEncadeada.py

A lista é dupla. A classe No guarda ant, valor e prox. A classe ListaDupla guarda comeco e ultimo, as duas pontas.

inserir_inicio cria o nó, aponta o prox dele para o antigo começo e, se a lista estava vazia, esse nó também vira o ultimo. Senão, o antigo começo passa a ter ant apontando para o novo. inserir_final faz o espelho disso usando ultimo. As duas inserções só mexem em ponteiros, então são O(1).

buscar sai de comeco e anda por prox até achar o valor ou acabar a lista. No pior caso olha todo mundo, O(n). remover chama buscar e, se achar, religa os vizinhos: o ant.prox pula para o prox, e o prox.ant volta para o ant. Se o nó era a ponta, comeco ou ultimo é atualizado. Achar custa O(n). Soltar o nó, depois de achado, é O(1).

listar_inicio_fim anda com prox. listar_fim_inicio anda com ant. Os dois juntam os valores com " -> ".

No main eu insiro 10, 20, 30, 40 e 50 no final e removo 30. A impressão fica:

10 -> 20 -> 40 -> 50
50 -> 40 -> 20 -> 10

Depois eu monto outra lista com os números de 1 a 100000 e meço buscar(10), buscar(50000) e buscar(99990). A busca do 10 é a mais curta porque buscar sempre começa no primeiro nó. Meio e fim andam bem mais nós. O tempo é perf_counter em volta só do buscar.

cd /app/Exercicio3
python3 ListaDuplaEncadeada.py

Perguntas

1. Qual é a complexidade da inserção no início?
O(1). Eu não percorro a lista, só atualizo comeco e o ant do antigo primeiro nó.

2. Qual é a complexidade da inserção no final?
O(1). A lista já guarda ultimo, então não preciso andar até o fim para encaixar o novo nó.

3. Qual é a complexidade da busca?
O(n). buscar só sabe ir para a frente, a partir de comeco.

4. Qual é a complexidade da remoção?
O(n) para achar o valor. A troca dos ponteiros, com o nó já em mãos, é O(1).

5. Qual é uma vantagem da lista duplamente encadeada em relação à lista simplesmente encadeada?
Dá para andar nos dois sentidos, como nas duas listagens. Se eu já estou num nó, também consigo removê-lo sem procurar o anterior, porque ant está no próprio nó.

6. Qual é o custo dessa vantagem?
Cada nó guarda mais um ponteiro. Gasta mais memória e, na inserção e na remoção, eu tenho que acertar os dois lados, não só o prox.

Exercício 4

Arquivo: Exercicio4/Labirinto.py

O labirinto vem de /dados/labirinto.txt. Para ver o arquivo:

cat /dados/labirinto.txt

é parede, espaço é caminho livre, S é o começo e E é a saída. Eu leio cada linha com rstrip só do \n, para não apagar os espaços do caminho, e guardo como lista de caracteres.

achar varre linha e coluna até achar o símbolo. Se S ou E não existir, o programa avisa "Inicio ou saida nao encontrados." e para.

andar é a busca recursiva. A ordem dos passos é cima, baixo, esquerda, direita. Se a casa atual já é E, devolvo True na hora. Senão marco a casa no set vistos. Para cada vizinho eu checo se está dentro do mapa, se não é # e se ainda não foi visitado. Aí chamo andar de novo.

Se a chamada volta False, aquele lado não chega na saída. A função segue para o próximo passo. Esse retorno é o backtracking: desisto daquele ramo e tento outro a partir da casa de onde eu vim. Se alguma chamada volta True, e a casa atual é espaço, eu troco por ".". S não vira ponto, porque a marca só acontece quando o caractere é espaço. E também não é reescrito, porque a função retorna assim que vê E, antes de marcar.

O caminho não é obrigatoriamente o mais curto. É o primeiro que essa ordem de direções consegue fechar. No fim o programa imprime o labirinto e "Caminho encontrado!" ou "Nenhum caminho encontrado!".

Com o set, cada casa entra no máximo uma vez. Se o labirinto tem L linhas e C colunas, o tempo fica limitado por O(L × C). A memória extra também, por causa do set e da pilha de chamadas.

cd /app/Exercicio4
python3 Labirinto.py

Perguntas

1. Onde ocorre a recursividade no algoritmo?
Dentro de andar, na chamada andar(lab, nl, nc, vistos) para o vizinho.

2. Onde ocorre o backtracking?
Quando essa chamada volta False. A casa atual não segue naquele vizinho e o for tenta a próxima direção.

3. Por que as posições visitadas precisam ser armazenadas?

Para não entrar de novo na mesma casa. Sem isso o caminho pode voltar para onde já esteve e girar sem fim.

4. O que poderia acontecer se uma posição pudesse ser visitada indefinidamente?
A recursão não pararia. O programa estouraria o limite de chamadas.

5. O que acontece com o custo do algoritmo quando existem muitas possibilidades de caminho?
Ele pode olhar mais casas antes de achar E. Com vistos, cada casa acessível é processada uma vez, então o custo continua preso ao tamanho do labirinto, não ao número de voltas no mesmo corredor.

Exercício 5

É a mochila 0/1. Cada item entra no máximo uma vez. O arquivo /dados/mochila.txt traz na primeira linha a quantidade de itens e a capacidade. As linhas seguintes trazem nome, peso e valor. Dá para ver o arquivo com:

cat /dados/mochila.txt

Os dois programas leem esse arquivo do mesmo jeito e têm que chegar no mesmo valor máximo. O que muda é como chegam lá.

Parte A. Arquivo: Exercicio5/ParteA_Exer5.py

A função mochila recebe a lista, o índice do item e a capacidade que ainda cabe. Toda entrada soma 1 em chamadas, inclusive quando acaba os itens ou a capacidade chega a 0, que devolve 0.

Se o peso do item passa da capacidade restante, eu sigo só sem ele. Se cabe, calculo as duas opções: sem o item, e com o item (somo o valor e diminuo o peso da capacidade). Fico com o maior dos dois. O perf_counter mede só essa busca.

A saída imprime "SOLUCAO RECURSIVA", o valor máximo, o número de chamadas e o tempo. Como cada item abre duas decisões, o tempo fica em torno de $O(2^n)$. Vários caminhos diferentes caem no mesmo par (índice, capacidade) e a função calcula tudo de novo.

cd /app/Exercicio5
python3 ParteA_Exer5.py

Parte B. Arquivo: Exercicio5/ParteB_Exer5.py

mochila_pd monta uma tabela com uma linha a mais que a quantidade de itens e uma coluna a mais que a capacidade (o 0 entra). tab[i][c] é o melhor valor usando os i primeiros itens com capacidade c.

Para cada item e para cada capacidade eu somo 1 em estados. Se o peso não cabe, copio o valor de cima, tab[i - 1][c]. Se cabe, comparo ficar sem o item com valor + tab[i - 1][c - peso] e guardo o maior. Por isso a quantidade de estados é a quantidade de casas dessa varredura: itens × (capacidade + 1).

A saída imprime "PROGRAMACAO DINAMICA", o valor máximo, os estados e o tempo. A complexidade é O(n × C), com n itens e C a capacidade. A tabela gasta O(n × C) de memória. A recursão gasta O(n) na pilha, mas refaz subproblema.

python3 ParteB_Exer5.py

Os dois valores máximos têm que bater. A programação dinâmica costuma ser bem mais rápida nesse caso porque cada combinação de item e capacidade é feita uma vez só. A razão entre os tempos depende da máquina e do que mais estiver rodando no container.

Perguntas

1. Quais subproblemas são calculados repetidamente na versão recursiva?
Os pares de índice do item com a mesma capacidade que ainda resta. Caminhos diferentes da recursão chegam nesse mesmo par e a função mochila calcula de novo.

2. Como a programação dinâmica evita essas repetições?

Ela guarda o melhor resultado em tab[i][c] e não recalcula aquela casa. Cada estado entra uma vez no contador.

3. Qual é a complexidade aproximada da solução recursiva?
$O(2^n)$, porque cada item pode ser pego ou deixado de fora.

4. Qual é a complexidade da solução utilizando programação dinâmica?
$O(n \times C)$. n é a quantidade de itens e C é a capacidade.

5. Existe algum custo adicional ao utilizar programação dinâmica?
Sim. A tabela ocupa memória $O(n \times C)$. Em troca, o tempo deixa de explodir com as chamadas repetidas.

Exercício 6

Não fiz.